

FIAP

FIAP

SLIDER

Gestão da Qualidade & Métricas de avaliação da Qualidade

Conteúdo

- Gestão da Qualidade de Software;
- Métricas de Avaliação da Qualidade de Software;
- Ferramentas de Medição da Qualidade do Código.

Gestão da Qualidade de Software

TQM, Total Quality Management ou Gestão Total da Qualidade

É um tipo de gerenciamento que envolve:

- Comprometimento de todos na organização;
- Dedicação a um nível alto de qualidade em todos os processos;
- Foco na satisfação do cliente.

Isso porque...



Gestão da Qualidade de Software

Cada letra do TQM carrega consigo um significado que sustenta cada uma dessas afirmações como veremos a seguir:

T – Total



Porque deve envolver toda a organização e cada aspecto do negócio

Q – Quality



Porque o tempo todo deve satisfazer as necessidades e expectativas do cliente

M - Management



Porque empodera a todos na organização para buscar e alcançar resultados de alta qualidade

Gestão da Qualidade de Software

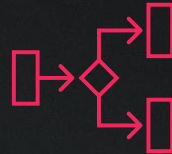
Os 8 Princípios do TQM:



- Foco no Cliente



- Todos os Colaboradores



- Centrado em Processos



- Processos Integrados

Gestão da Qualidade de Software



- Estratégia Sistemática
(regras e procedimentos Predefinidos, Objetivos e Testáveis)



- Melhoria Contínua



- Decisões Basedas em Fatos



- Comunicação



Gestão da Qualidade de Software

As ferramentas para aplicação do TQM ajudam a:

- Identificar problemas com a Qualidade.
- Analisar Impedimentos.
- Avaliar Dados.
- Identificar a Causa Raíz dos problemas.
- Medir Resultados.



Confira as ferramentas de TQM a seguir...

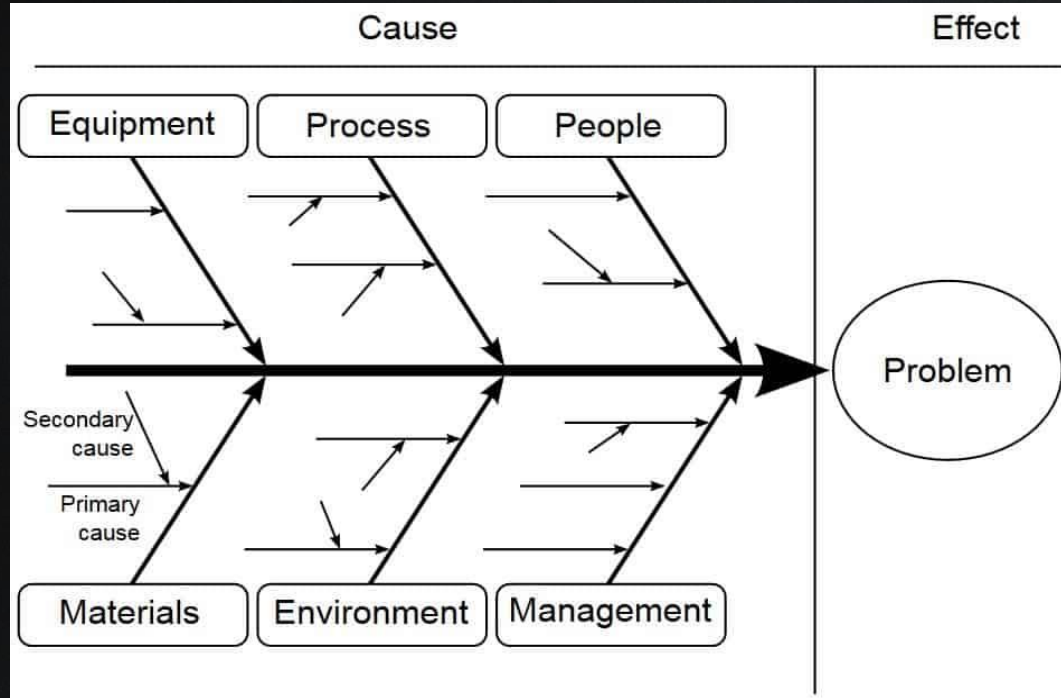
Gestão da Qualidade de Software

Folha de Verificação

Tipo de Defeito	Verificação	Total
Especificação		20
API - Unitário		13
API - Integração		30
Front - Componente		10
Front - Integração		5
Total		78

Gestão da Qualidade de Software

Diagrama Espinha de Peixe



Gestão da Qualidade de Software

Diagrama Espinha de Peixe

- **Método:** a metodologia para a realização do trabalho pode ser inadequada;
- **Material:** os materiais utilizados para o trabalho podem ser inadequados;
- **Mão-de-obra:** as pessoas podem enfrentar baixa produtividade, não ter a capacitação necessária para a realização do trabalho ou estar desestimuladas;
- **Máquinas:** as máquinas e as tecnologias utilizadas podem estar obsoletas ou apresentar problemas de funcionamento;
- **Medidas:** as métricas utilizadas para mensurar os resultados podem não ser as mais apropriadas para os objetivos dos projetos ou o controle desses resultados não está sendo feito corretamente;
- **Meio ambiente:** o meio ambiente diz respeito ao contexto de trabalho (como localização, atmosfera organizacional e condições de realização dos projetos).

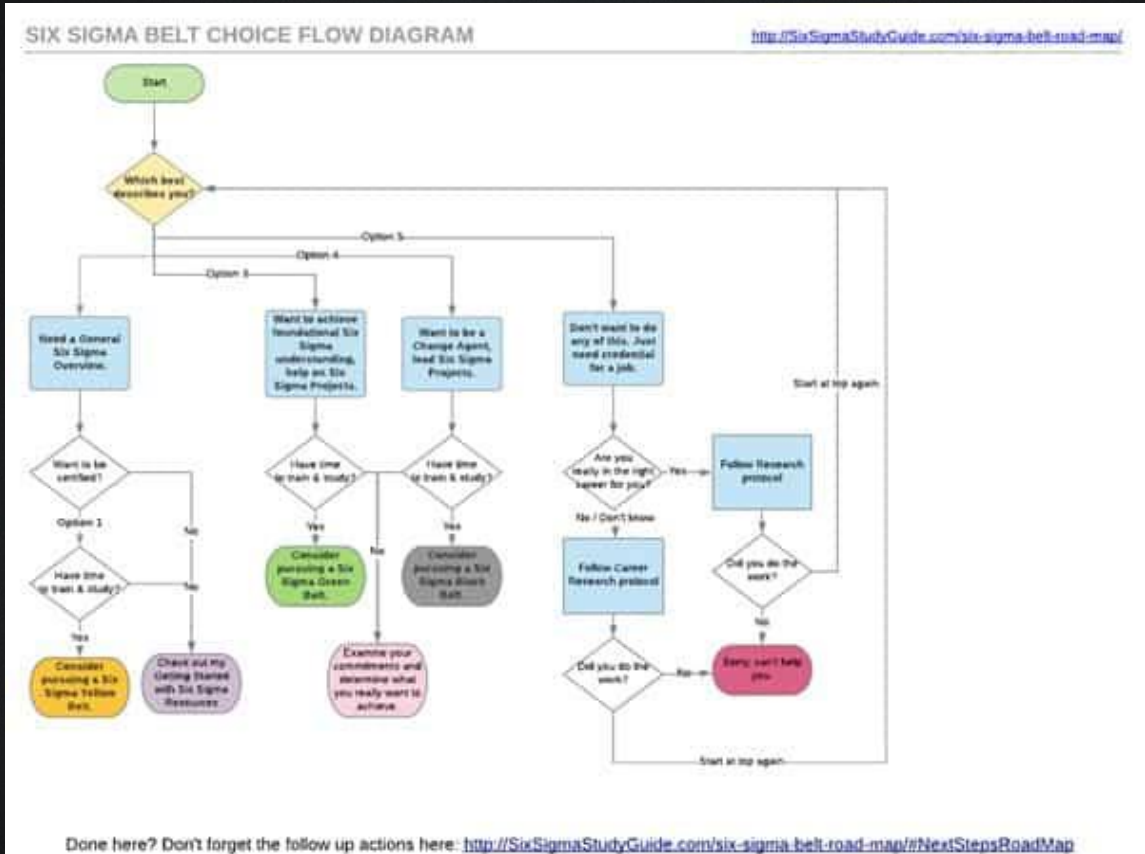
Gestão da Qualidade de Software

Fluxograma do Processo

Cria-se um AS IS

Para encontrar oportunidades
de melhoria e...

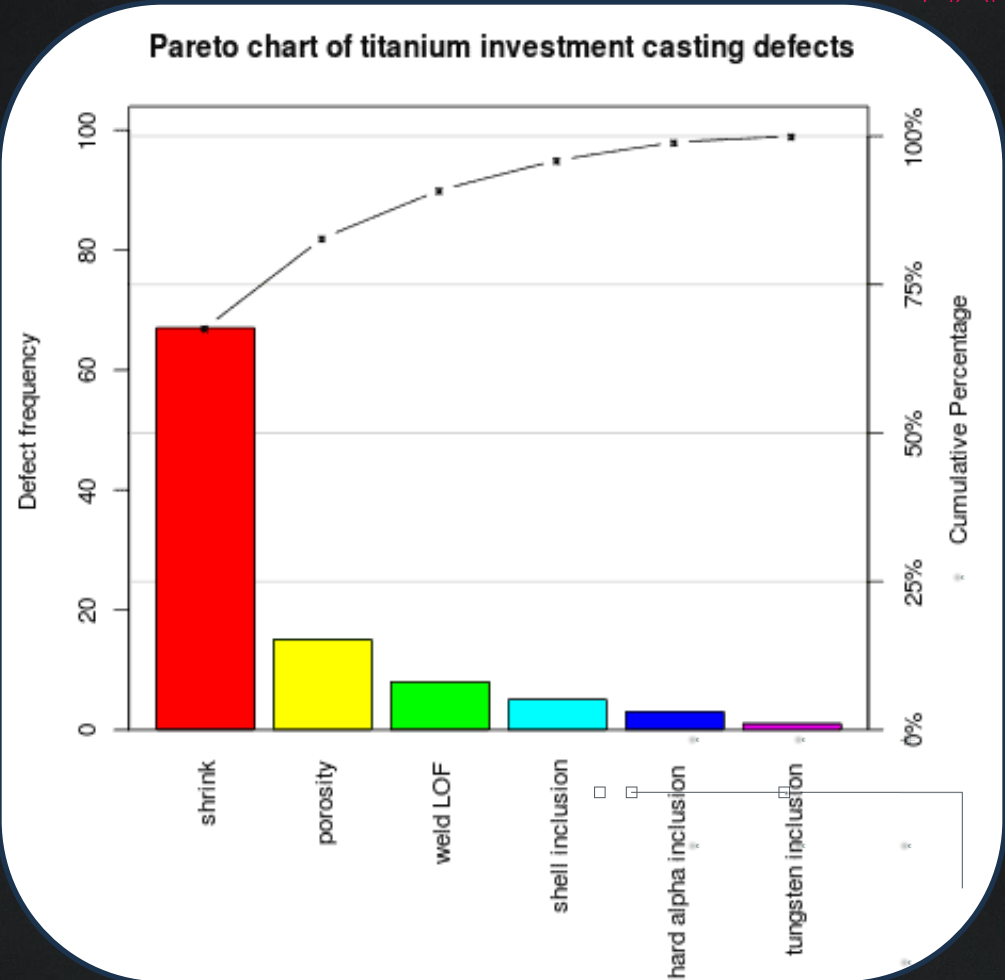
Criar e implementar um TO BE



Gestão da Qualidade de Software

Gráfico de Pareto

Baseado no princípio de Pareto:
80% dos efeitos vêm de
20% das causas.



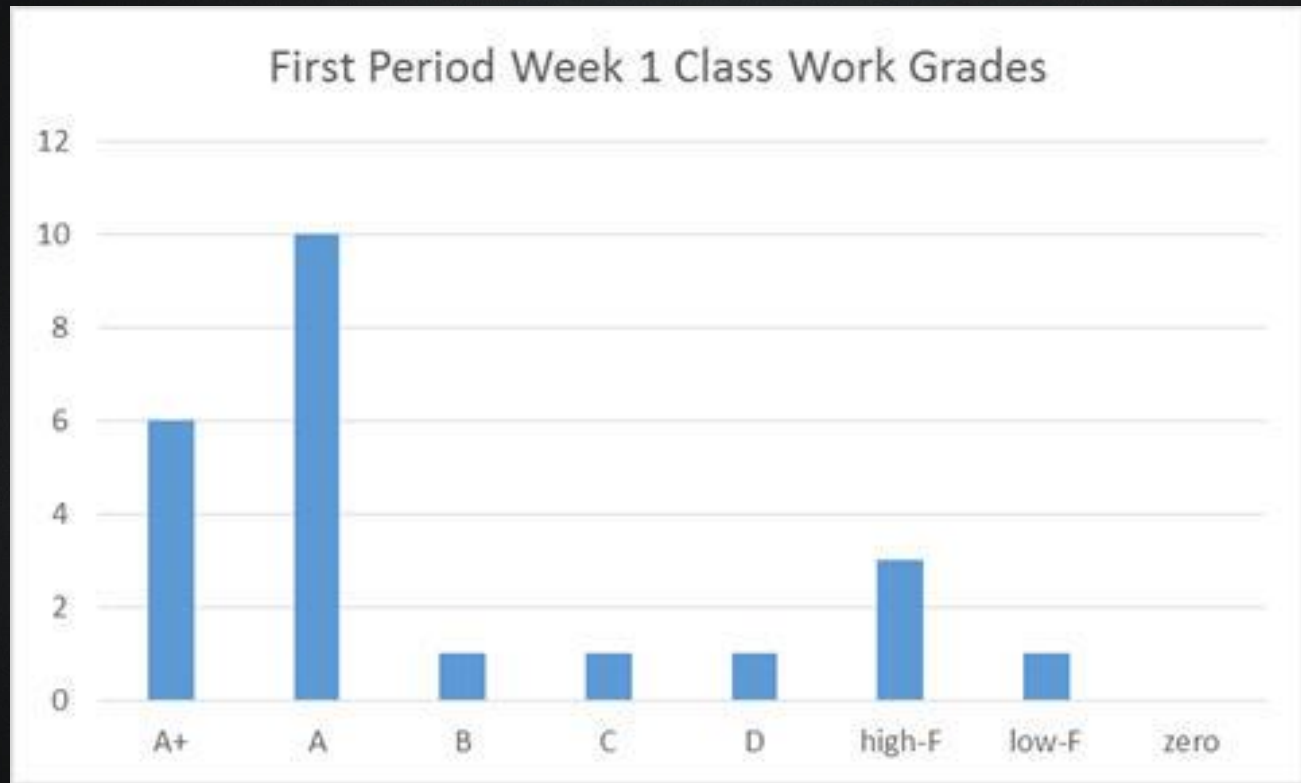
Gestão da Qualidade de Software

Histograma

Ajuda a identificar

- Padrões;
- Tendências;
- Anomalias.

E a tomar decisões.

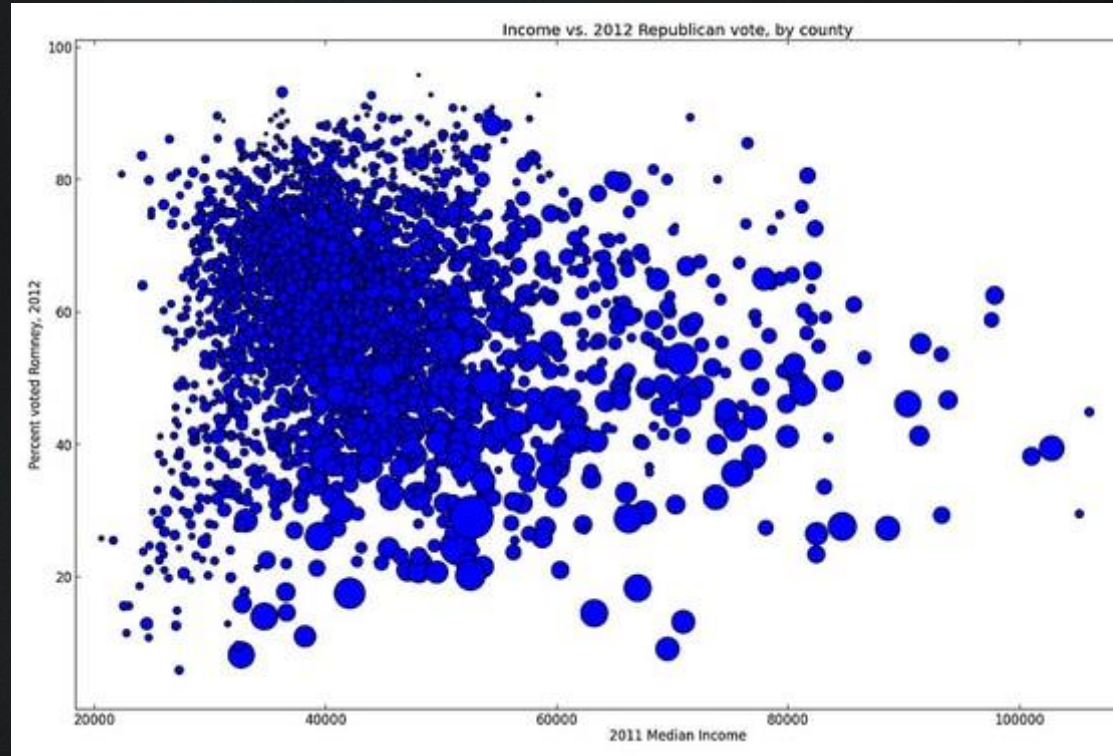


Gestão da Qualidade de Software

Diagrama de Dispersão

Representação gráfica da possível relação entre duas variáveis

Mostrando de forma gráfica os pares de dados numéricos e sua relação.

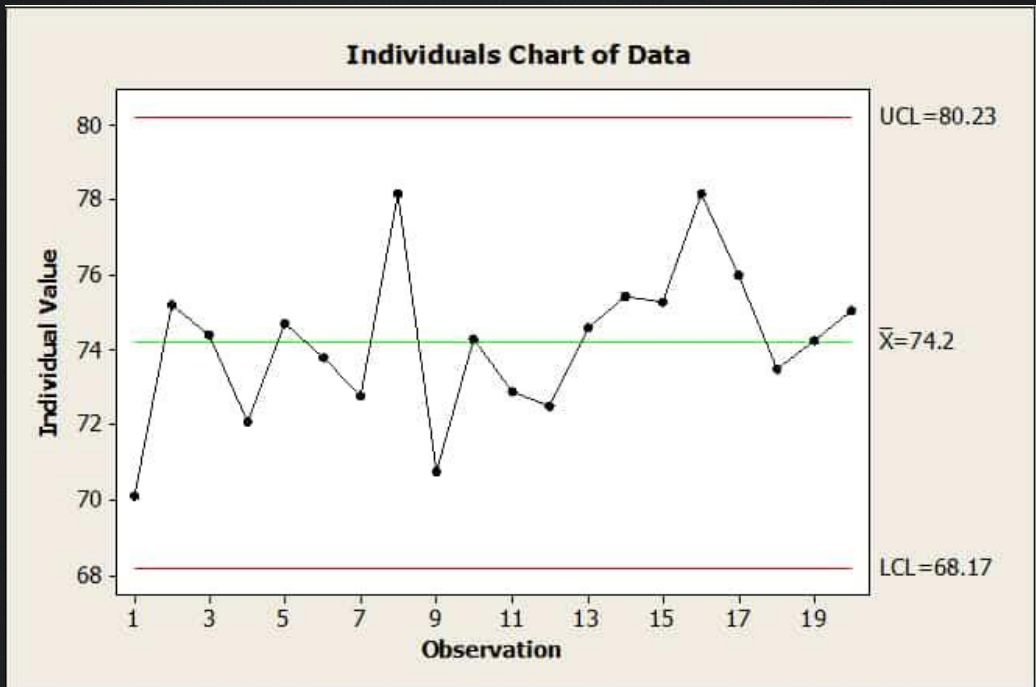


Gestão da Qualidade de Software

Gráfico de Controle

Variação ao longo do tempo.

Ou seja, observa as mudanças
em um processo dentro de
determinado período.



Modified from Figure 10.8, *Integrated Enterprise Excellence Volume III - Improvement Project Execution: A Management and Black Belt Guide for Going Beyond Lean Six Sigma and the Balanced Scorecard*, Forrest W. Breyfogle III, Bridgeway Books/Citius Publishing, Austin, TX, 2008.

Gestão da Qualidade de Software

Six Sigma

É baseada no conceito de “sigma”, que em estatística representa uma variação, ou desvio padrão, dentro de um sistema.

Tem como objetivos:

- Reduzir ao máximo os defeitos;
- Aumentar a qualidade nos produtos e serviços ;
- Diminuir a variabilidade dos processos produtivos.



Gestão da Qualidade de Software

Six Sigma x TQM

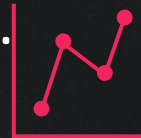
Compartilham algumas similaridades como:

- Monitoram a qualidade final;
- Visam diminuir o número de defeitos e erros;
- Ênfase em incluir todas as pessoas da organização;
- São centradas em processos.

E a diferença entre essas abordagens, é que:

TQM foca na melhoria de processos;

Six Sigma foca em diminuir variação.

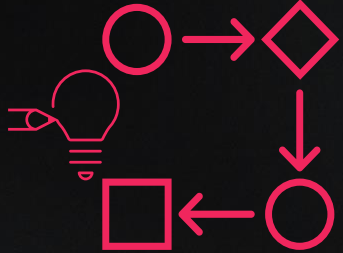


Gestão da Qualidade de Software

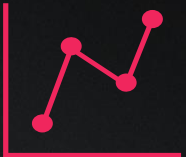
Os conceitos fundamentais de Seis Sigma são:



Sigma: Na metodologia Seis Sigma aplicada aos negócios, visa-se obter a maior qualidade nos processos produtivos, com o menor número de defeitos no processo.



DMAIC: significa Definir, Medir, Analisar, Melhorar e Controlar. Refere-se às etapas de um dos métodos mais utilizados na implementação da metodologia do Six Sigma.



Variabilidade: Na metodologia Seis Sigma, visa-se reduzir ao máximo a variabilidade para alcançar a excelência e a qualidade final esperada.

Gestão da Qualidade de Software



CTQ (Critical to Quality): é fundamental no Six Sigma e busca identificar as características cruciais dentro de um processo ou produto para atender as necessidades do cliente. Quando não alcançados, a qualidade do produto final é impactada, o que se reflete na experiência do cliente.

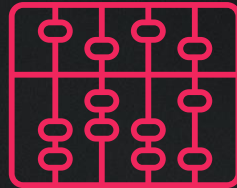


Melhoria contínua: filosofia do Six Sigma, que visa o aprimoramento contínuo com o objetivo de alcançar o maior nível de qualidade no processo produtivo.

Métricas de Avaliação da Qualidade de Software

Métricas de código fornecem aos desenvolvedores uma visão melhor do código que estão desenvolvendo.

A partir delas, os desenvolvedores podem entender quais tipos ou quais métodos devem ser trabalhados novamente ou testados com mais atenção.



Métricas de Avaliação da Qualidade de Software

Índice de manutenção

Índice entre 0 e 100 que representa o quão fácil é manter o código.

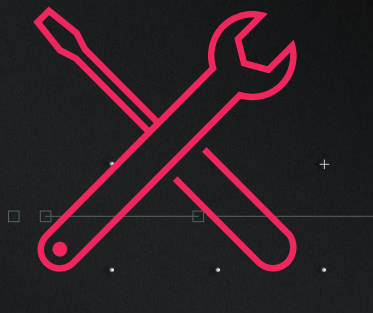
Quanto mais alto, melhor a capacidade de manutenção.

Algumas ferramentas classificam por cores, o que pode ajudar a identificar mais rapidamente pontos problemáticos em seu código. Ex.:

Verde está entre 20 e 100 e indica que o código tem boa capacidade de manutenção.

Amarelo está entre 10 e 19 e indica que o código pode ter manutenção moderada.

Vermelho fica entre 0 e 9 e indica baixa capacidade de manutenção.



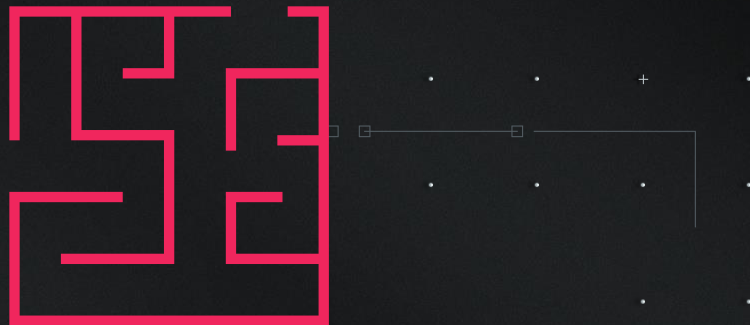
Métricas de Avaliação da Qualidade de Software

Complexidade ciclomática

- Mede a Complexidade Estrutural do código.

- É calculada a partir do número de caminhos de código diferentes no fluxo do programa.

- A partir dela podemos identificar se um programa que tem um fluxo de controle complexo e portanto requer mais testes para conseguir uma boa cobertura de código e menos manutenção.



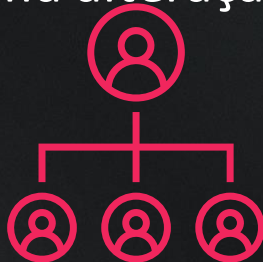
Métricas de Avaliação da Qualidade de Software

Profundidade de herança

- Número de classes diferentes que herdaram entre si até a classe base.
- É semelhante ao acoplamento de classe em que uma alteração em uma classe base pode afetar qualquer uma de suas classes herdadas.

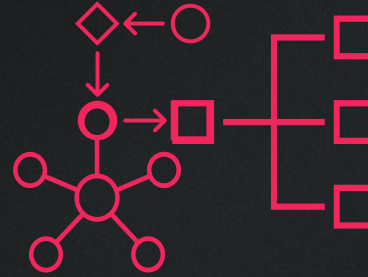
Quanto maior esse número, mais profunda a herança e maior o potencial de modificações de classe base que geram uma alteração interruptiva.

Quanto mais baixa, melhor
Quanto mais alta, pior.



Métricas de Avaliação da Qualidade de Software

Acoplamento de classe



- Acoplamento a classes exclusivas por meio de parâmetros, variáveis locais, tipos de retorno, chamadas de método, instâncias genéricas ou de modelo, classes base, implementações de interface, campos definidos em tipos externos e decoração de atributo.

Um bom design de software determina os tipos e métodos que devem ter alta coesão e baixo acoplamento.

O alto acoplamento indica um design difícil de reutilizar e manter devido a diversas interdependências em outros tipos.

Métricas de Avaliação da Qualidade de Software

Linhas do código-fonte

Número exato de linhas de código-fonte presentes no arquivo de origem, incluindo linhas em branco.

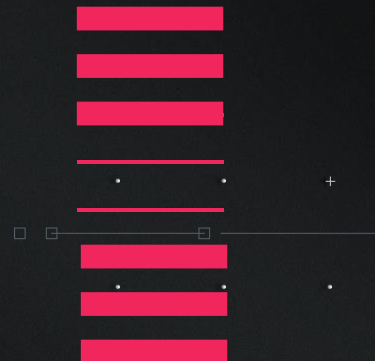


Linhas de código executável

Número aproximado de operações ou linhas de código executáveis.

É uma contagem de número de operações no código executável.

O valor normalmente é uma correspondência próxima à métrica anterior, Linhas de Código.



Hands-On Ferramentas de Métricas do Código

Vamos calcular métricas de código via IDEs usando Plugins

Abra seu projeto Java no VSCode ou IntelliJ

Siga as instruções de cada IDE

Hands-On Ferramentas de Métricas do Código

Usando o VS Code Counter no VSCode

- Em "Extensions" pesquise por "VS Code Counter" by Kentaro Ushiyama
- Clique em "Install"
- Reinicie o VScode
- Digite "Ctrl+Shift+P" com o VSCode reaberto
- Ou apenas digite ">" na pesquisa principal
- E então digite e pesquise por "VSCodeCounter"
- Selecione a opção "VSCodeCounter: Count lines in workspace."



Hands-On Ferramentas de Métricas do Código

Usando o IntelliJ Statistic Metrics no IntelliJ

- Digite "Ctrl+Alt+S" e Selecione a opção "Plugins"
- Em Marketplace pesquise por "Statistic" by Tomas Topinka
- Clique em "Install"
- Reinicie o IntelliJ
- Com o IntelliJ reaberto clique na aba "Statistic"
- Então clique no botão "Refresh"
- Clique na aba Java



Refências

Total Quality Management and Six Sigma, Posted by Ted Hessing @ <https://sixsigmastudyguide.com/total-quality-management-and-six-sigma/>

O que é Six Sigma e como aplicar a metodologia na prática. Posted by Victoria Carneiro @ <https://pm3.com.br/blog/six-sigma/> <https://pm3.com.br/blog/diagrama-de-ishikawa/> <https://ferramentasdaqualidade.org/diagrama-de-dispersao/>

Valores de métrica de código, Collaborated by [Mikejo5000](#) [ghogen](#) [GitHubber17](#) [Stacyrch140](#) [damabe](#) [Saisang JKirsch1](#) [DCtheGeek](#) [rickvdbosch](#) [ploeh](#) [mijacobs](#) [gewarten](#) @ <https://learn.microsoft.com/pt-br/visualstudio/code-quality/code-metrics-values?view=vs-2022>

<https://marketplace.visualstudio.com/items?itemName=uctakeoff.vscode-counter>
<https://plugins.jetbrains.com/plugin/4509-statistic/versions/stable>
<https://medium.com/@benkaddourmed54/understanding-and-optimizing-code-a-comprehensive-guide-5cfa963c94a9>

<https://www.sonarsource.com/learn/static-code-analysis-using-sonarqube/#installing-sonarqube-community-build>

<https://www.sonarsource.com/products/sonarlint/#modal=what-is-sonarlint>

FIAP